

Các mô hình liên tục tuyến tính (Linear Continuous Models)

Tối ưu hoá – AI203DV01
(Optimization)
Vũ Đình Khôi

Nội dung

- Mixing
- Blending
- Project Management
- Multi-Stage Models
- Pattern Classification

- Chapter 02. Serge Kruk. (2018). *Practical Python AI Projects: Mathematical Models of Optimization Problems with Google OR-Tools*. Apress

**I find that I don't understand things unless I try
to program them.**



Donald E. Knuth

Professor Emeritus at Stanford University

If you want to **master something**, teach it



-- **Richard Feynman**

■ Linear programming (LP)

- Quy hoạch tuyến tính (QHTT) là bài toán tối ưu hóa (optimization), trong đó hàm mục tiêu (objective function) và các điều kiện ràng buộc (constraints) đều là tuyến tính.

■ Continuous

- Tất cả các biến/thuộc tính (variables) có giá trị liên tục

■ Ví dụ, hàm mục tiêu tuyến tính có dạng

$$c_1x_1 + c_2x_2 + \dots + c_nx_n$$

■ Không phải (phi tuyến)

$$x_1^2, x_4^3, \sin(x), e^{x_3}, x_1 \cdot x_2, \text{ or } |x|$$

■ Tập các quan hệ ràng buộc biểu diễn dưới dạng ma trận a, vector/mảng b

$$a_{i1}x_{i1} + a_{i2}x_{i2} + \dots + a_{in}x_{in} \geq b_i, \text{ or } \leq b_i, \text{ or } = b_i$$

Giới thiệu

■ Lưu ý về các ràng buộc

- Linear solver chỉ chấp nhận các ràng buộc (constraint) $\leq$, $=$, $\geq$
- Nếu các ràng buộc $>$ (hoặc $<$) sẽ chuyển thành $\geq$, hoặc $\leq$

■ Ví dụ

- $a > b$ sẽ thành $a \geq b + 1$
- $a < b$ sẽ thành $a \leq b - 1$

■ Linear continuous models (các mô hình liên tục tuyến tính)

- Tất cả các biến đều liên tục (All variables are continuous) $\rightarrow$ các biến quyết định (decision variables) có thể nhận giá trị nguyên (integral) hoặc phân số (fractional).
- Tất cả các ràng buộc đều tuyến tính (All constraints are linear).
- Hàm mục tiêu tuyến tính (The objective function is linear).

■ Đơn giản nhất để biểu diễn và giải

Mixing: Problem

- Dạng/lớp bài toán kết hợp (mixing problem)
- Bài toán kinh điển tính phần ăn/thực đơn (diet problem)
 - Từ yêu cầu của quân đội Mỹ (thập niên 1930 – 1940)
 - Do George Stigler nghiên cứu và giải đầu tiên với phương pháp heuristic, sau đó Jack Laderman (NIST) giải với phương pháp simplex
(https://developers.google.com/optimization/lp/stigler_diet)
 - Mô hình tuyến tính của bài toán này gồm 9 phương trình (equations) và 77 biến (unknowns) → thời điểm đó, tốn khá nhiều thời gian để giải.
- Phát biểu bài toán tổng quát
 - Cho danh sách các thực phẩm/khẩu phần ăn/món ăn (food) cùng với hàm lượng dinh dưỡng (nutritional content) và chi phí (cost) → kết hợp các thực phẩm sao cho chi phí tối thiểu, nhưng vẫn đảm bảo các yêu cầu dinh dưỡng cần thiết.

Mixing: Problem

■ Dữ kiện được trình bày trong bảng sau

- F_i : Các món ăn (foods) như pizza, ramen noodles, cupcakes, chips...
- N_i : hàm lượng dinh dưỡng (nutritional content) từng phần ăn (serving) như calories, protein, calcium, vitamin A...
- Min, Max: số lượng phần ăn tối thiểu/tối đa mỗi món ăn (number of servings)
- Cost: chi phí mỗi phần ăn
- 2 dòng min/max cuối cùng là tổng hàm lượng dinh dưỡng tối thiểu/tối đa

	N0	N1	N2	N3	Min	Max	Cost
F0	606	563	665	23	7	17	9.06
F1	68	821	83	72	6	27	8.42
F2	28	70	916	56	1	36	9.47
F3	121	429	143	38	14	26	6.97
F4	60	179	818	46	9	35	4.77
Min	5764	28406	48157	1642			
Max	15446	76946	82057	6280			

Mixing: Constructing a model

■ Decision variables (cần xác định)

- Các biến f_i tương ứng với số lượng phần ăn mỗi món

■ Dữ kiện chi phí

- c_i chi phí ứng với mỗi phần ăn

■ Objective function (Hàm mục tiêu)

$$\min \sum_i c_i \times f_i$$

■ Constraints (các tập ràng buộc)

- Số lượng tối thiểu/lower (l_i) và tối đa/upper mỗi món ăn (u_i)

$$l_i \leq f_i \leq u_i$$

- Tổng giá trị từng loại dinh dưỡng của các tất cả các phần ăn

$$a_j \leq \sum_i N_{ji} \times f_i \leq b_j$$

Mixing: Constructing a model

					l_i	u_i	
f_i	N0	N1	N2	N3	Min	Max	Cost
F0	606	563	665	23	7	17	9.06
F1	68	821	83	72	6	27	8.42
F2	28	70	916	56	1	36	9.47
F3	121	429	143	38	14	26	6.97
F4	60	179	818	46	9	35	4.77
a_i Min	5764	28406	48157	1642			
b_i Max	15446	76946	82057	6280			

Matrix N

Mixing: Constructing a model

Table 2-1. *Example of Data and Solution for the Diet Problem*

	N0	N1	N2	N3	Min	Max	Cost	Solution
F0	606	563	665	23	7	17	9.06	17.0
F1	68	821	83	72	6	27	8.42	7.47
F2	28	70	916	56	1	36	9.47	6.11
F3	121	429	143	38	14	26	6.97	14.0
F4	60	179	818	46	9	35	4.77	35.0
Min	5764	28406	48157	1642				f_i
Max	15446	76946	82057	6280				
Sol	14775	28406	48157	3413			539.37	

Mixing: Executable model

```
1  """ Bài toán tối ưu dinh dưỡng (diet problem)
2  dạng mixing (Linear Continuous Models)
3  (diet_problem.py)
4  """
5  from ortools.linear_solver import pywraplp
6
7
8  # N ma trận 2 chiều/bảng chứa thông tin món ăn và dinh dưỡng
9  def solve_diet(N):
10     solver = pywraplp.Solver('Diet', pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
11
12     # Số lượng món ăn và dinh dưỡng
13     n_foods = len(N) - 2
14     # trừ 3 cột cuối cùng chứa MinNutrient, MaxNutrient, Cost
15     n_nutrients = len(N[0]) - 3
16
17     # Vị trí (index) các cột MinServing (l), MaxServing (u), Cost
18     f_min, f_max, f_cost, = n_nutrients, n_nutrients + 1, n_nutrients + 2
19     # Vị trí (index) các dòng MinNutrient (a), MaxNutrient (b)
20     n_min, n_max = n_foods, n_foods + 1
```

Mixing: Executable model

```
22 # f là mảng gồm các biến quyết định trong khoảng
23 # MinServing (l) và MaxServing (u)
24 f = [solver.NumVar(N[i][f_min], N[i][f_max], 'f[%i]' % i)
25       for i in range(n_foods)]
26 print("Number of variables =", solver.NumVariables())
27
28 # Các ràng buộc (constraints)
29 for j in range(n_nutrients):
30     solver.Add(solver.Sum([f[i] * N[i][j]
31                             for i in range(n_foods)]) >= N[n_min][j])
32     solver.Add(solver.Sum([f[i] * N[i][j]
33                             for i in range(n_foods)]) <= N[n_max][j])
34 print("Number of constraints =", solver.NumConstraints())
35
36 solver.Minimize(solver.Sum([f[i] * N[i][f_cost]
37                             for i in range(n_foods)]))
38
39 status = solver.Solve()
40
```

Mixing: Executable model

```
42 # Trả về mảng chứa giá trị tối ưu của các biến quyết định f[i]
43 if status == solver.OPTIMAL:
44     return [f[i].solution_value() for i in range(n_foods)]
45 elif status != solver.OPTIMAL:
46     print("The problem does not have an optimal solution!")
47     if status == solver.FEASIBLE:
48         print("A potentially suboptimal solution was found.")
49     else:
50         print("The solver could not solve the problem.")
51     return None
52
```

Mixing: Executable model

```
54 def main():
55     N = [[606, 563, 665, 23, 7, 17, 9.06],
56          [68, 821, 83, 72, 6, 27, 8.42],
57          [28, 70, 916, 56, 1, 36, 9.47],
58          [121, 429, 143, 38, 14, 26, 6.97],
59          [60, 179, 818, 46, 9, 35, 4.77],
60          [5764, 28406, 48157, 1642, ],
61          [15446, 76946, 82057, 6280, ]]
62
63     f = solve_diet(N)
64     print("Servings of each food: \n", f)
65
```

```
>>> %Run diet_problem.py
Number of variables = 5
Number of constraints = 8
Servings of each food:
[17.0, 7.473907368545185, 6.113172148920027, 14.0, 35.0]
```

Mixing: Variations

- Một số biến thể/dạng khác (variations) của bài toán

- Tối đa lợi nhuận (profit), thay vì tối thiểu chi phí (cost)
- Ràng buộc giữa các biến quyết định, ví dụ

$$f_3 \geq f_2$$

- Ràng buộc giữa các dữ kiện, ví dụ

$$\sum_i N_{i3} f_i \geq 2 \sum_i N_{i4} f_i$$

- Các bài toán với cấu trúc như “diet problem” được gọi chung là “product mix problem”

Mixing: Variations

- Các bài toán với cấu trúc như “diet problem” được gọi chung là “product mix problem”, cấu trúc trừu tượng (abstract) của bài toán

Table 2-2. *Abstract Structure of Product Mix Problems*

		Components			Availabilities		Cost
		C_1	...	C_n	Min	Max	
Products	P_1	99	...	99	99	99	99
	...	...	...	...	...	...	...
	P_m	99	...	99	99	99	99
Demands	Min	99	...	99	99	99	99
	Max	99	...	99	99	99	99
Price		99	...	99	99	99	99

Blending problem

- Dạng/lớp bài toán hỗn hợp
- Bài toán điều chế/pha trộn xăng/dầu, từ dầu thô → thành phẩm tinh chế.
 - Cho bảng thông tin dầu thô với hàm lượng octan, số lượng (thùng) và chi phí

***Table 2-3.** Example of Raw Gasolines for the Blending Problem*

Gas	Octane	Availability	Cost
R0	99	782	55.34
R1	94	894	54.12
R2	84	631	53.68
R3	92	648	57.03
R4	87	956	54.81
R5	97	647	56.25
R6	81	689	57.55
R7	96	609	58.21

Blending problem

- Bảng các thành phẩm tinh chế gồm hàm lượng octan, nhu cầu tối thiểu/tối đa và giá thành
 - Giả định, hàm lượng octan thành phẩm = tỷ trọng các octan dầu thô tương ứng. Ví dụ nếu trộn 40% của octane 80 và 60% của octane 90, ta được:

$$40\% \times 80 + 60\% \times 90 = 32 + 54 = 86$$
- Xây dựng mô hình (model) để xác định cực đại lợi nhuận (profit)

Table 2-4. *Example of Refined Gasolines for the Blending Problem*

Gas	Octane	Min. Demand	Max. Demand	Price
F0	88	415	11707	61.97
F1	94	199	7761	62.04
F2	90	479	12596	61.99

Blending problem

- Kết quả cần tính là bảng dưới đây (bảng G)

Table 2-5. Complete Solution to the Blending Problem

	F0	F1	F2	Barrels	Cost
R0	542.5		239.5	782.0	43275.88
R1		894.0		894.0	48383.28
R2	631.0			631.0	33872.08
R3		648.0		648.0	36955.44
R4	704.41	251.59		956.0	52398.36
R5		647.0		647.0	36393.75
R6	449.5		239.5	689.0	39651.95
R7	50.93	558.07		609.0	35449.89
Barrels	2378.33	2998.67	479.0		
Price	147385.32	186037.28	29693.21		36735.18

G_{ij} : số lượng
dầu thô i góp
phần sản xuất
thành phẩm j

Blending problem

- Câu hỏi “Phải sản xuất bao nhiêu mỗi loại thành phẩm?”
 - Đúng (correct) nhưng chưa đầy đủ (incomplete)
- “Làm cách nào để pha trộn dầu thô thành thành phẩm?”
 - Câu hỏi đúng (right question), nhưng chưa phù hợp với mô hình đại số (algebraic model)

Blending problem: Constructing a model

- G_{ij} : số lượng (thùng) dầu thô loại i dùng để pha trộn thành phẩm j
 - Ví dụ: $G_{51} = 250 \rightarrow$ cần 250 thùng dầu thô R_5 để sản xuất thành phẩm F_1
- Do đó, ta có tổng sản lượng dầu thô mỗi loại i (cộng theo hàng, bảng G)

$$R_i = \sum_j G_{ij} \quad \forall i$$

- Tổng thành phẩm mỗi loại j (cộng theo cột, bảng G)

$$F_j = \sum_i G_{ij} \quad \forall j$$

- Tất nhiên

$$\sum_i R_i = \sum_j F_j$$

Blending problem: Constructing a model

■ Hàm mục tiêu (objective function)

$$\max \sum_j F_j p_j - \sum_i R_i c_i$$

■ Các ràng buộc (constraint)

- u_i là sản lượng dầu thô i (cột Available)

$$0 \leq R_i \leq u_i \quad \forall i$$

- a_j, b_j : nhu cầu sản lượng tối thiểu/tối đa thành phẩm j

$$a_j \leq F_j \leq b_j \quad \forall j$$

- O_i : hàm lượng octan dầu thô i , o_j là hàm lượng octan thành phẩm j

$$\sum_i O_i G_{ij} = F_j o_j \quad \forall j$$

Blending problem: Constructing a model

```
1 from ortools.linear_solver import pywraplp
2 from my_or_tools import ObjVal, SolVal
3
4
5 # C, D: bảng thông tin về dầu thô (raw) và thành phẩm (refined)
6 def solve_gas(C, D):
7     s = pywraplp.Solver('Gas blending problem',
8                         pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
9
10    nR, nF = len(C), len(D) # Số lượng dầu thô và thành phẩm
11    Roc, Rmax, Rcost = 0, 1, 2 # vị trí các cột trong bảng C
12    Foc, Fmin, Fmax, Fprice = 0, 1, 2, 3 # vị trí các cột trong bảng D
13
14    # Các decision variables của bảng G (mảng 2 chiều)
15    G = [[s.NumVar(0.0, 10000, '') for j in range(nF)] for i in range(nR)]
16
17    # Các decision variables của bảng R và F
18    R = [s.NumVar(0, C[i][Rmax], '') for i in range(nR)]
19    F = [s.NumVar(D[j][Fmin], D[j][Fmax], '') for j in range(nF)]
20
```

Blending problem: Constructing a model

```
21 # Các equations về tổng dầu thô Ri và tổng thành phẩm Fj
22 for i in range(nR):
23     s.Add(R[i] == sum(G[i][j] for j in range(nF)))
24 for j in range(nF):
25     s.Add(F[j] == sum(G[i][j] for i in range(nR)))
26 for j in range(nF):
27     s.Add(F[j] * D[j][Foc] == s.Sum([G[i][j] * C[i][Roc] for i in range(nR)]))
28
29 Cost = s.Sum(R[i] * C[i][Rcost] for i in range(nR))
30 Price = s.Sum(F[j] * D[j][Fprice] for j in range(nF))
31 s.Maximize(Price - Cost)
32
33 status = s.Solve()
34 return status, ObjVal(s), SolVal(G)
```

Blending problem: Constructing a model

```
37 def main():
38     # Bảng thông tin dầu thô (crude)
39     C = [[99, 782, 55.34],
40          [94, 894, 54.12],
41          [84, 631, 53.68],
42          [92, 648, 57.03],
43          [87, 956, 54.81],
44          [97, 647, 56.25],
45          [81, 689, 57.55],
46          [96, 609, 58.21]]
47
48     # Bảng thông tin thành phẩm (refined)
49     D = [[88, 415, 11707, 61.97],
50          [92, 479, 12596, 61.99],
51          [94, 199, 7761, 62.04],
52          [90, 479, 12596, 61.99]]
53
54     status, value, G = solve_gas(C, D)
55     print('Value of objective function (profit): {:.2f}'.format(value))
```

Blending problem: Constructing a model

```
57 for i in range(len(G)):  
58     for j in range(len(G[i])):  
59         print('{:0.1f} \t'.format(G[i][j]), end=' ')  
60     print()  
61
```

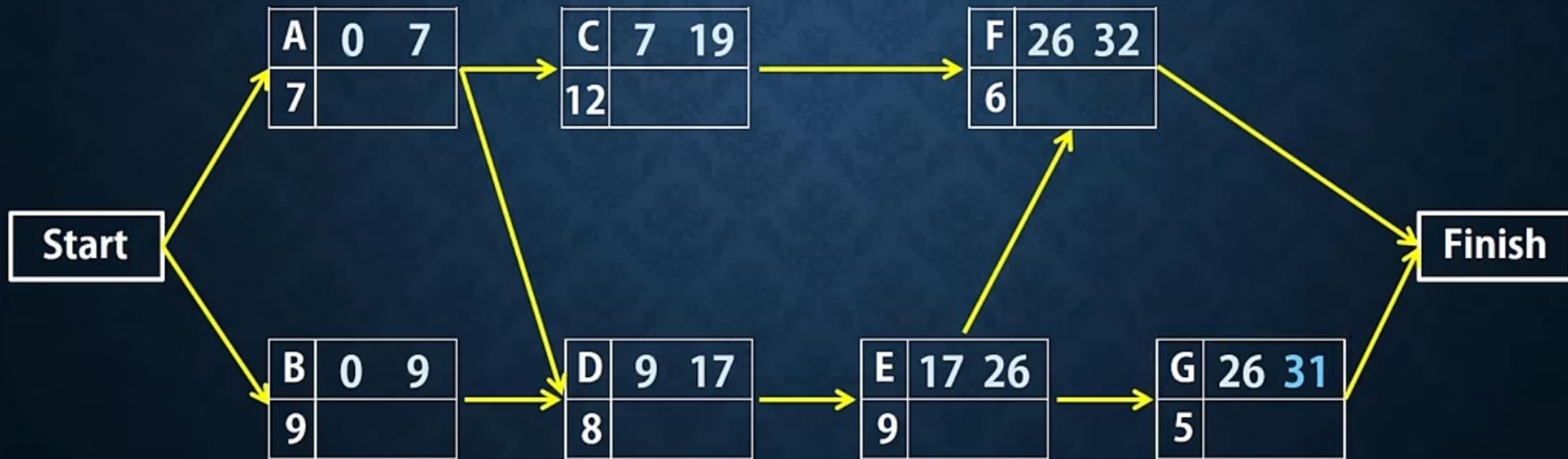
```
>>> %Run gas_blend.py  
Value of objective function (profit): 36722.40  
489.3    292.7    0.0    0.0  
56.7     0.0    632.0    205.3  
631.0     0.0     0.0     0.0  
0.0       0.0    648.0     0.0  
682.3     0.0     0.0    273.7  
0.0       0.0    647.0     0.0  
359.4    186.3    143.3     0.0  
0.0       0.0    609.0     0.0
```

- Bài toán quản lý dự án
 - Cho tập hợp các công việc (tasks): T
 - Mỗi công việc cần thời gian/số ngày hoàn thành (duration) D và tập con (subset) các công việc cần phải thực hiện trước (preceding tasks)
 - Có thể thực hiện một số task song song
- Xác định thời điểm bắt đầu (starting time) mỗi công việc để tổng thời gian hoàn thành dự án là tối thiểu
- Ví dụ: xem bảng slide kế tiếp

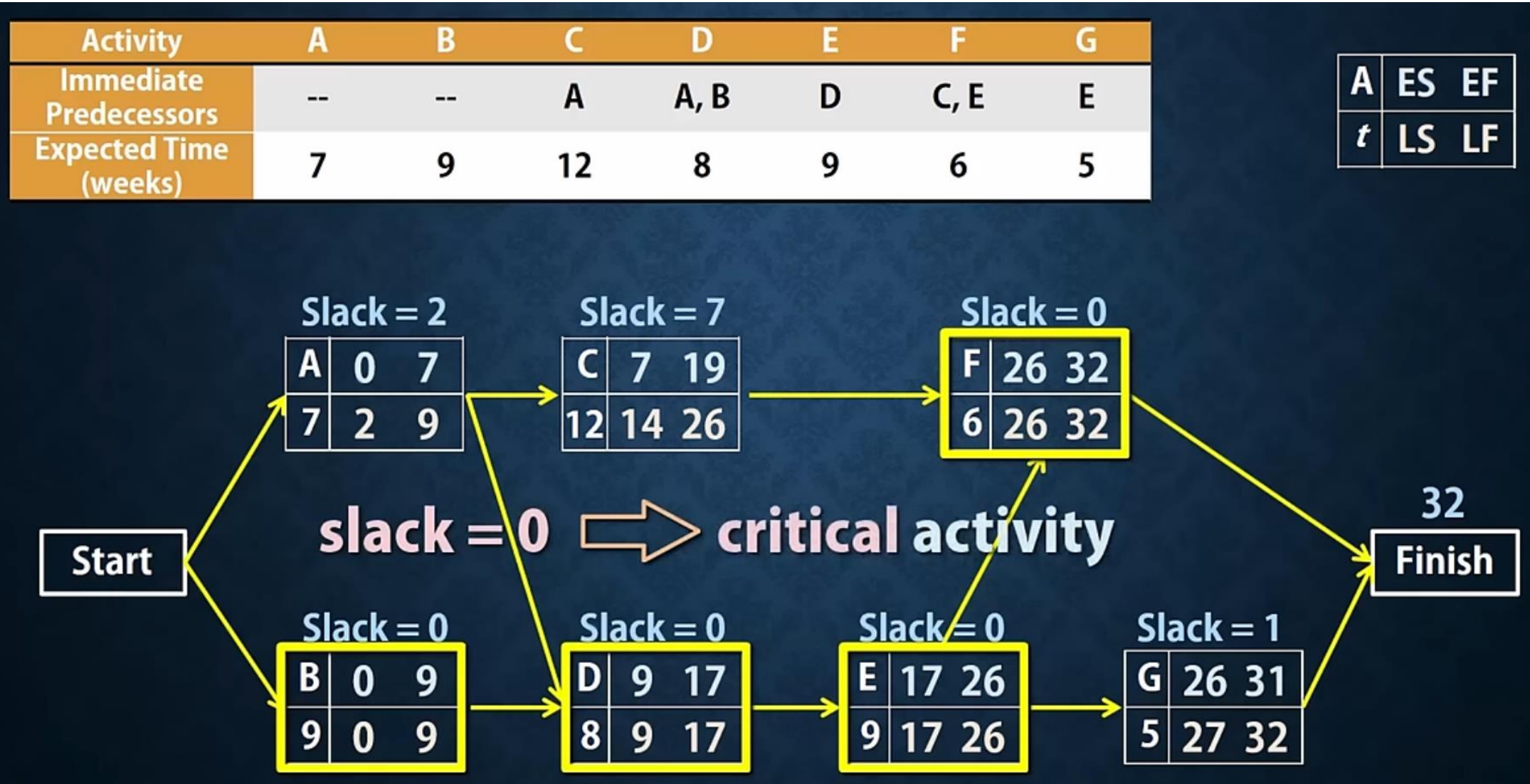
Project management

Activity	A	B	C	D	E	F	G
Immediate Predecessors	--	--	A	A, B	D	C, E	E
Expected Time (weeks)	7	9	12	8	9	6	5

A	ES	EF
t	LS	LF



Project management



Project management

■ Lưu ý:

- $i = 0, 1, 2, \dots$ là công việc (task) thứ 0, 1, 2 (cột 1)
- T_i : tập con các công việc phải thực hiện trước (precedence) công việc i (cột 3)
- t_i : thời điểm bắt đầu (starting time) của công việc thứ i (các giá trị cần tính?)

Table 2-7. *Example of Project Management Tasks*

Task	Duration	Preceding Tasks
0	3	{ }
1	6	{ 0 }
2	3	{ }
3	2	{ 2 }
4	2	{ 1 2 3 }
5	7	{ }
6	7	{ 0 1 }
7	5	{ 6 }
8	2	{ 1 3 7 }
9	7	{ 1 7 }
10	4	{ 7 }
11	5	{ 0 }

Project management: constructing model

- Các biến quyết định (decision variables)

$$t_i \geq 0 \quad \forall i \in T$$

- Các ràng buộc (constraints)

$$t_j + D_j \leq t_i \quad \forall j \in T_i; \forall i \in T$$

- Ý nghĩa: thời điểm bắt đầu công việc i (t_i) sẽ sau thời điểm hoàn thành trễ nhất của một công việc j nào trong tập con T_i (chính là thời điểm bắt đầu công việc j cộng với thời gian hoàn thành công việc j)

- Hàm mục tiêu (objective function)

- Tối thiểu giá trị t cho tập các ràng buộc

$$t_i + D_i \leq t \quad \forall i \in T$$

Project management

```
7 # D: bảng mô tả công việc (xem hình project_management.png)
8 def solve_project_management(D):
9     s = pywraplp.Solver('Project management problem',
10                        pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
11     n = len(D) # Số lượng công việc
12     # Thời gian hoàn thành tối đa
13     # (xem như các công việc được xếp lịch tuần tự/nối tiếp nhau)
14     days_max = sum(D[i][1] for i in range(n))
15     # Các biến quyết định
16     t = [s.NumVar(0, days_max, 't[%i]' % i) for i in range(n)]
17     total = s.NumVar(0, days_max, 'Total')
18     # Các ràng buộc
19     for i in range(n):
20         s.Add(t[i] + D[i][1] <= total)
21         for j in D[i][2]: # các công việc tiên quyết trong tập con {...}
22             s.Add(t[j] + D[j][1] <= t[i])
23     s.Minimize(total)
24     status = s.Solve()
25     return status, ObjVal(s), SolVal(t)
```

Project management

```
28 def main():
29     # D[i] = [task i, duration, {preceding tasks of task i}]
30     D = [[0, 3, {}],
31          [1, 6, {0}],
32          [2, 3, {}],
33          [3, 2, {2}],
34          [4, 2, {1, 2, 3}],
35          [5, 7, {}],
36          [6, 7, {0, 1}],
37          [7, 5, {6}],
38          [8, 2, {1, 3, 7}],
39          [9, 7, {1, 7}],
40          [10, 4, {7}],
41          [11, 5, {0}]]
42
```

Project management

```
43 status, value, t = solve_project_management(D)
44 print('Value of objective function (total time): {:.2f}'.format(value))
45 print('Schedule:')
46 print('Task\t', end=' ')
47 n = len(t)
48 for i in range(n):
49     print('{:d}\t'.format(i), end=' ')
50 print('\nStart\t', end=' ')
51 for i in range(n):
52     print('{:.0f}\t'.format(t[i]), end=' ')
53 print('\nEnd \t', end=' ')
54 for i in range(n):
55     print('{:.0f}\t'.format(t[i] + D[i][1]), end=' ')
```

Value of objective function (total time): 28.00

Schedule:

Task	0	1	2	3	4	5	6	7	8	9	10	11
Start	0	3	0	3	9	0	9	16	26	21	24	23
End	3	9	3	5	11	7	16	21	28	28	28	28

■ Lưu ý

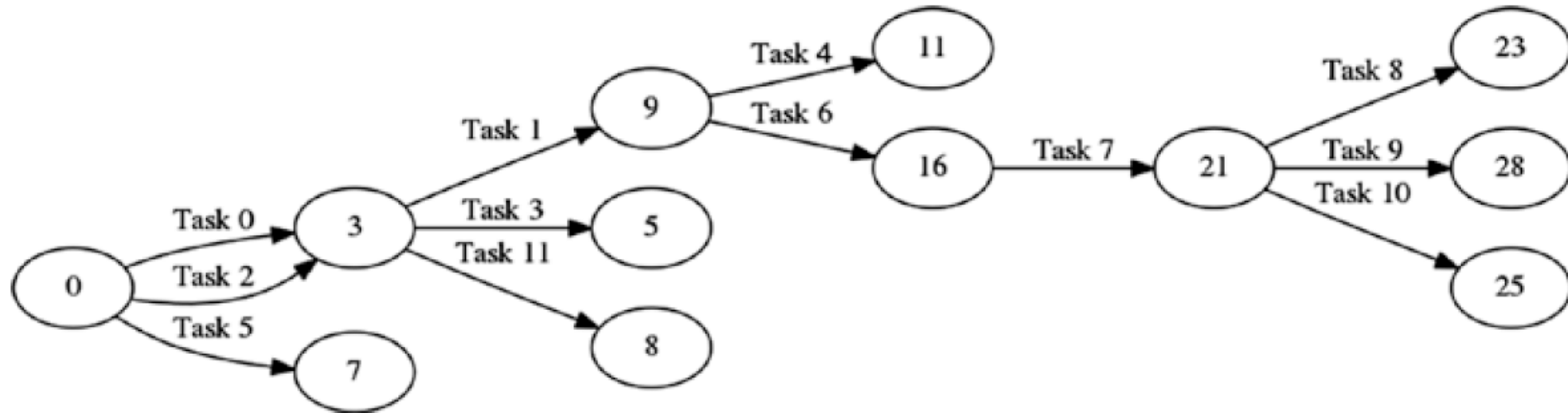
- Tùy thuộc vào solver được sử dụng, nghiệm của bài toán có thể khác chút (nhưng kết quả cuối cùng là tổng thời gian hoàn thành dự án không thay đổi)

Table 2-9. An Alternate Optimal Solution to the Project Management Problem

Task	0	1	2	3	4	5	6	7	8	9	10	11
Start	0	3	0	3	9	0	9	16	21	21	21	3
End	3	9	3	5	11	7	16	21	23	28	25	8

Project management

- Các node trên đồ thị là thời điểm bắt đầu [hoặc kết thúc] của task i nào đó
 - Trực quan, ta thấy tập con các task {0, 1, 2, 6, 7, 9} là các công việc tới hạn (critical tasks) → nếu các task này trễ hạn sẽ dẫn đến thời gian hoàn thành dự án trễ hạn (delayed).
- Làm sao xác định được các critical task?
 - Sẽ trở lại trong phần Shortest Path của Linear Network Models (section 4.4.3 Chapter 4)



Project management: variations

■ Bài tập

- Sinh viên thay đổi mô hình (model) để được kết quả sau đây

Table 2-9. An Alternate Optimal Solution to the Project Management Problem

Task	0	1	2	3	4	5	6	7	8	9	10	11
Start	0	3	0	3	9	0	9	16	21	21	21	3
End	3	9	3	5	11	7	16	21	23	28	25	8

Project management: variations

■ Xác định thời điểm bắt đầu các công việc sớm nhất có thể?

- Hàm mục tiêu

$$\min \sum_i t_i$$

- Để ý 2 mục tiêu tối thiểu thời gian hoàn thành dự án (minimize completion time) và các task bắt đầu sớm nhất (minimize earliest starting time) là nhất quán nhau (coherent).

■ Bài toán đa mục tiêu (multiple objectives)

- Thường các mục tiêu độc lập (independent), trường hợp xấu nhất có thể trái ngược nhau (contradictory).

Project management: minmax variations

- Khi chúng ta muốn tối thiểu giá trị cực đại của một/nhiều hàm (minimize the maximum of functions)

$$\min_x \max_{i \in T} \sum_j a_{i,j} \times x_j$$

- Chuyển bài toán sang hàm mục tiêu min t
- Cùng với ràng buộc

$$\sum_j a_{i,j} x_j \leq t \quad \forall i \in T$$

- (Rõ ràng, t là giá trị max của hàm và mục tiêu là tìm t tối thiểu)
- Bài toán maxmin tương tự, nhưng các bài toán minmin/maxmax khó hơn
 - Sẽ trở lại trong phần Maximax và Minimin chương Advanced Techniques (section 7.2.4 chapter 7)

Project management: variations

■ Bài toán giá trị tuyệt đối (Absolute Value Problems)

- Tìm giá trị tối ưu (optimal) x cho hàm mục tiêu

$$\min_x \left| \sum_j c_j x_j \right|$$

■ Chuyển sang hàm mục tiêu

$$\min t$$

- Cùng với 2 ràng buộc

$$\begin{aligned} \sum_j c_j \times x_j &\leq t \quad \forall i \in T \\ - \left(\sum_j c_j \times x_j \right) &\leq t \quad \forall i \in T \end{aligned}$$

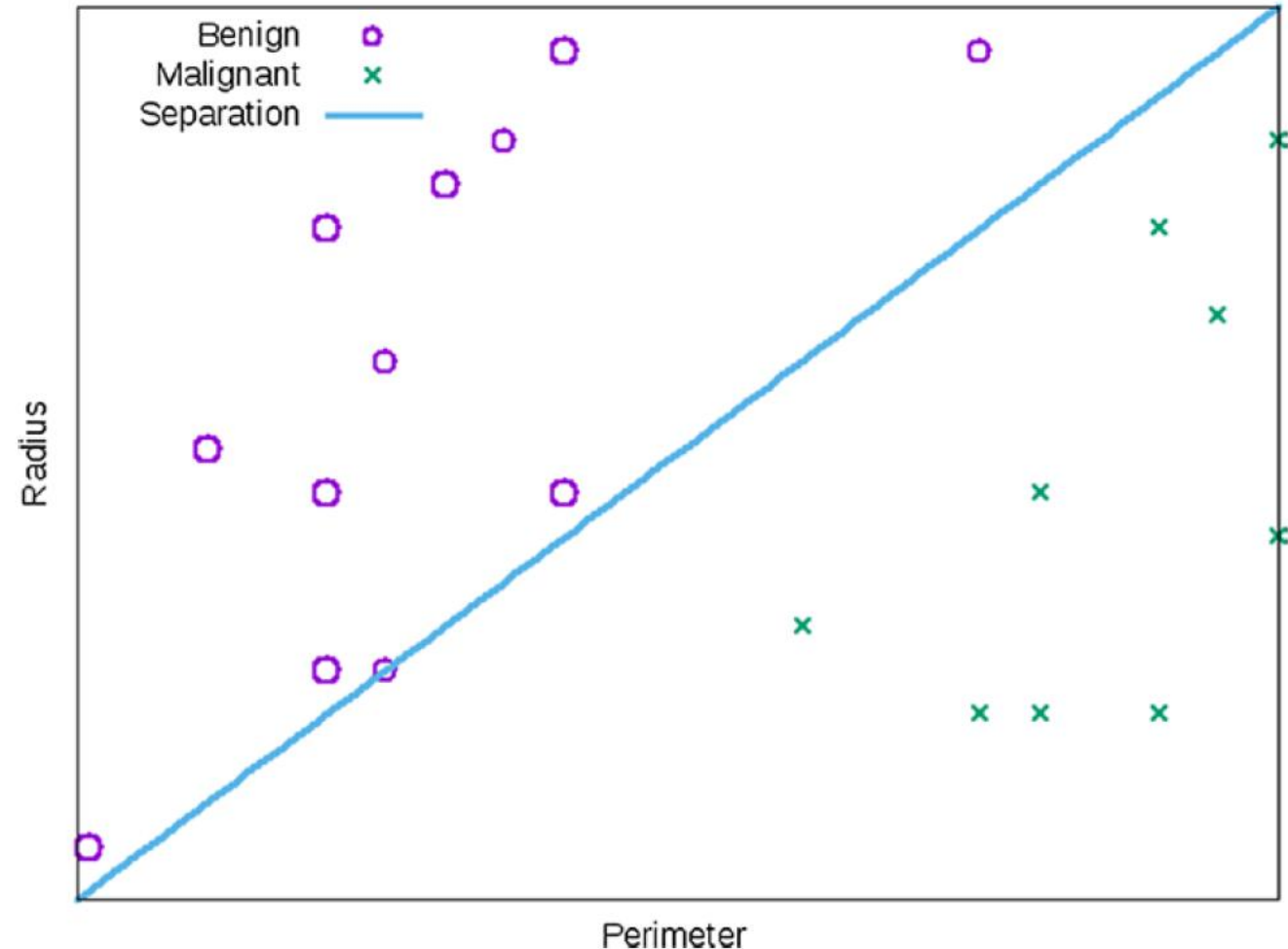
- Sẽ trở lại trong section 3.2 (Curve fitting) chapter 3 (Hidden Linear Continuous Models)

Pattern Classification

- Bài toán phân lớp nhị phân
 - Email: legitimate hay spam
 - Bio-cell: benign hay malignant
- Bài toán phân loại tế bào lành tính/ác tính dựa trên quan sát diện tích và chu vi tế bào
 - Thu thập thông tin/hình ảnh tế bào dưới kính hiển vi
 - Phân loại bởi chuyên gia → tập huấn luyện
 - Tế bào mới → phân nhóm/loại?

Pattern Classification

- Giả định tồn tại một đường chia cắt (phân tuyến) dạng:
 $a_1x_1 + a_2x_2 = a_0$ (hoặc
phương trình đường thẳng
 $y = ax + b$)
chia tập huấn luyện thành 2 lớp:
 - Class A: các điểm nằm một bên (one side) phân tuyến
 - Class B: các điểm nằm bên còn lại (other side) của phân tuyến



Pattern Classification: constructing a model

- Đối với không gian n-chiều, phương trình phân tuyến (hyperplane) có dạng

$$\sum_{i=1}^n a_i x_i = a_0$$

- Với 1 điểm dữ liệu mới x' trong tập thử (test), nếu:
 - $a_1 x_1 + a_2 x_2 > a_0 \rightarrow$ điểm x' nằm bên phân tuyến A $\rightarrow x'$ thuộc lớp A
 - $a_1 x_1 + a_2 x_2 < a_0 \rightarrow$ điểm x' nằm bên phân tuyến B $\rightarrow x'$ thuộc lớp B
 - Lưu ý: không có khái niệm trên/dưới phân tuyến, nếu ta chuyển bất đẳng thức thành $a_1 x_1 + a_2 x_2 - a_0 > 0$ (hoặc < 0) $\rightarrow$ chỉ có khái niệm cùng dấu hay trái dấu.
 - Bài toán tìm vector các hệ số a_i
- 2 bất đẳng thức nghiêm ngặt (strict inequalities) trên có thể được điều chỉnh (scaled) để tăng khoảng cách (gap) giữa các điểm đến phân tuyến
 - nghĩa là các điểm trong tập huấn luyện nằm xa đường phân tuyến một chút

Pattern Classification: constructing a model

▪ Để đơn giản, công việc của chúng ta là xác định một vector a sao cho

- Mọi điểm x' thuộc lớp A thỏa (giả định $a_1x_1 + a_2x_2 > a_0$)

$$\sum_i a_i x'_i \geq a_0 + 1$$

- Và một điểm x'' của lớp B thỏa

$$\sum_i a_i x''_i \leq a_0 - 1$$

Vì các ràng buộc trong linear programming chỉ có thể $\geq$, $=$, $\leq$ nên ta chuyển $x > a$ thành $x \geq (a + 1)$ nghĩa là x cách xa a khoảng cách 1 đơn vị chẳng hạn.

▪ Đối với lớp A, B bất đẳng thức được viết lại

$$0 \geq a_0 + 1 - \sum_i a_i x'_i$$

$$0 \geq \sum_i a_i x''_i - a_0 + 1$$

Pattern Classification: constructing a model

- Cho biến dương (positive variable) y'_i cho mỗi điểm dữ liệu lớp A

$$y' \geq a_0 + 1 - \sum_i a_i x'_i \text{ và } y' \geq 0$$

- Tương tự với mỗi điểm dữ liệu lớp B

$$y'' \geq \sum_i a_i x''_i - a_0 + 1 \text{ và } y'' \geq 0$$

- Vậy công việc giải bài toán phân lớp trở thành giải bài toán tối ưu (tối thiểu) các giá trị y' , y'' về zero

$$\min \sum_{i \in A} y'_i + \sum_{i \in B} y''_i$$

Pattern Classification: code

```
8 # A là tập dữ liệu nằm một bên đường phân tuyến (hyperplane)
9 # sao cho  $a_0 * x_0 + a_1 * x_1 + \dots + a_{(n-1)} * x_{(n-1)} - a_n > 0$ 
10 # B ngược với A. Do đó, khi test dữ liệu mẫu nêu tính ra giá trị  $> 0$  thì nó thuộc A
11 def solve_binary_classification(A, B):
12     n = len(A[0]) # Số lượng biến quyết định
13     ma, mb = len(A), len(B)
14     a_min, a_max = -99, 99
15     s = pywraplp.Solver('Binary pattern classification',
16                         pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
17     # Các biến quyết định. Lưu ý các cận trên/dưới có thể thay đổi
18     # vector a trong khoảng -99 đến 99 có thể kết quả không chính xác nhưng tính nhanh
19     ya = [s.NumVar(0, a_max, '') for _ in range(ma)]
20     yb = [s.NumVar(0, a_max, '') for _ in range(mb)]
21     # Lưu ý vector a có thêm một phần tử ở cuối là  $a_n$  nên gồm  $n + 1$  phần tử
22     # (theo công thức trên slides là  $a_0$ )
23     a = [s.NumVar(a_min, a_max, '') for _ in range(n + 1)]
```

Pattern Classification: code

```
25 # Các ràng buộc. Lưu ý: a[n] chính là a0 trong slide/textbook
26 k = 1 # khoảng cách tối thiểu từ mỗi điểm đến hyperplane
27 for i in range(ma):
28     s.Add(ya[i] >= a[n] + k - s.Sum(a[j] * A[i][j] for j in range(n)))
29 for i in range(mb):
30     s.Add(yb[i] >= s.Sum(a[j] * B[i][j] for j in range(n)) - a[n] + k)
31
32 # Hàm mục tiêu
33 A_gap = s.Sum(ya[i] for i in range(ma))
34 B_gap = s.Sum(yb[i] for i in range(mb))
35 s.Minimize(A_gap + B_gap)
36 status = s.Solve()
37 return status, ObjVal(s), SolVal(a)
38
```

Pattern Classification: code

```
40 def main():
41     # Các tập dữ liệu tập A, B
42     # A = [[1, 2], [2, 4], [4, 9], [5, 6]]
43     # B = [[4, 1], [5, 2], [6, 4], [8, 9]]
44
45     # Phát sinh tập dữ liệu mẫu
46     means = [[2, 4], [5, 3]]
47     cov = np.array([[.6, .2], [.2, .6]]) # ma trận hiệp phương sai
48     N = 100
49     np.random.seed(7) # phát sinh dữ liệu ngẫu nhiên giống nhau giữa các lần chạy
50     A = np.random.multivariate_normal(means[0], cov, N)
51     B = np.random.multivariate_normal(means[1], cov, N)
52
53     status, obj_val, sol_val = solve_binary_classification(A, B)
54     print('Status =', status)
55     print('Objective value =', obj_val)
56     print('Solution =', sol_val)
57     # Vẽ đồ thị các điểm dữ liệu và đường phân chia
58     A = np.array(A)
59     B = np.array(B)
```

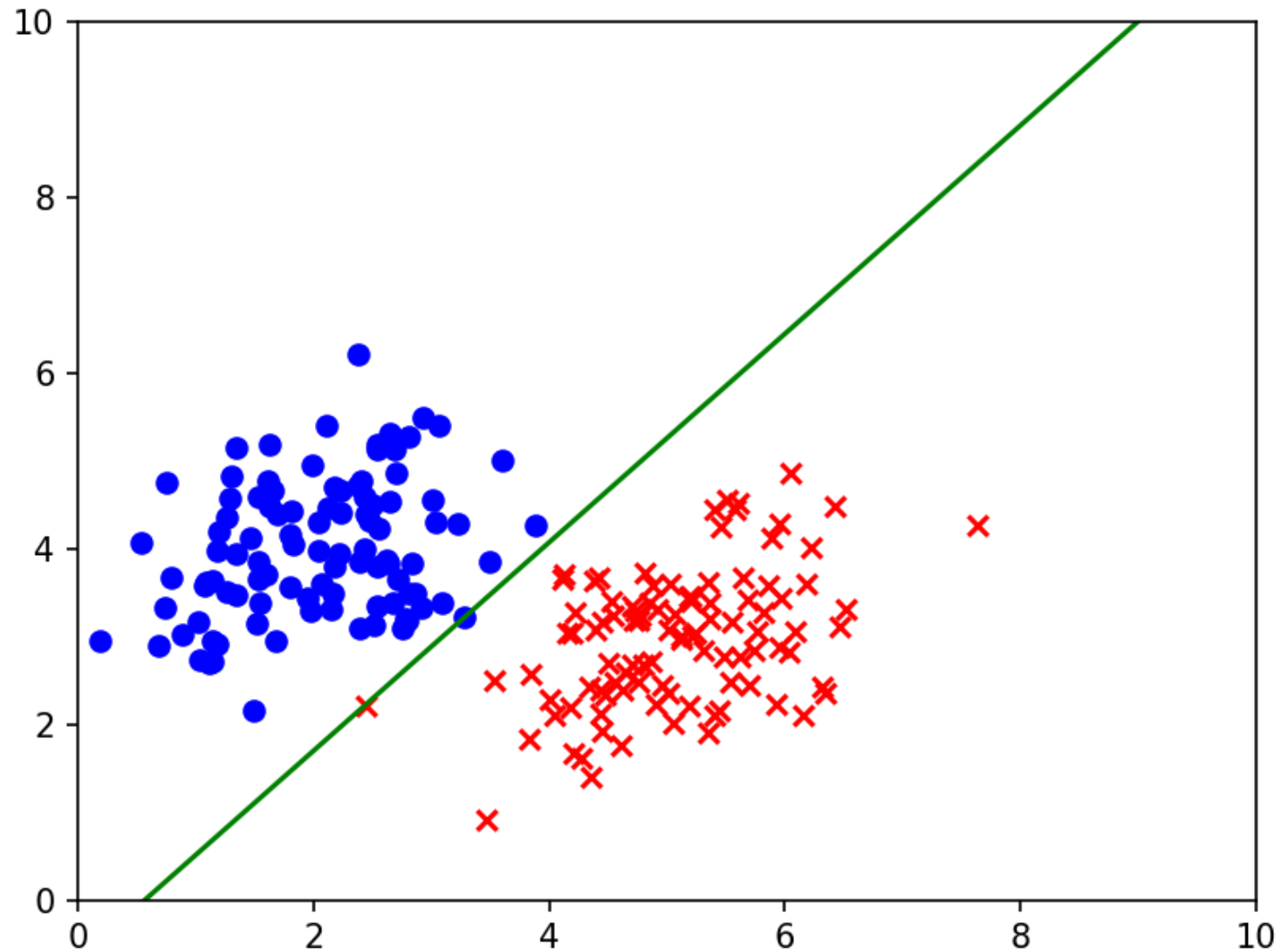


Pattern Classification: code

```
61 # Setting x-axis and y-axis limits
62 plt.xlim(0, 10), plt.ylim(0, 10)
63 plt.scatter(A[:, 0], A[:, 1], marker='o', color='blue')
64 plt.scatter(B[:, 0], B[:, 1], marker='x', color='red')
65 a = sol_val
66 x = np.linspace(0, 10, 100)
67 # Do pt hyperplane ban đầu dạng:  $a[0] * x_0 + a[1] * x_1 = a[2]$ 
68 #  $[x_0, x_1]$  tương ứng với giá trị  $(x, y)$  của phương trình  $y = ax + b$ 
69  $y = (a[2] - a[0] * x) / a[1]$ 
70 plt.plot(x, y, color='green')
71 plt.show()
72
```

```
>>> %Run pattern_classification.py
Status = 0
Objective value = 0.0
Solution = [-99.0, 83.38322506591398, -56.114224101300906]
```

Pattern Classification: code



- Sinh viên kiểm tra các điểm dữ liệu mới $(2, 2)$ và $(5, 4)$ thuộc lớp A hay B?

